SHIV NADAR INSTITUTION OF EMINENCE · CSD493 PROJECT-1 · B.TECH CSE

Behavioural-Diff Detection of Malicious Browser-Extension Updates

System approach and the machine-learning model — design, workflow, and evaluation

Team: Bind Pratap Singh (2310110084) · Suhani Deepak Agrawat (2310110717)

Advisor: Dr. Sweta Mishra · **Area:** Cybersecurity

Working title: extdrift · **Date:** September 2026

Summary. A browser extension that has been trusted once can be turned malicious by a single silent update. Existing store and browser checks are largely single-version and can be evaded by obfuscation, so a malicious update frequently reaches users before it is noticed. This document sets out our approach — running two consecutive versions of an extension under identical, deterministic conditions and scoring the *behavioural difference* between them — and describes how the machine-learning model is trained, how it makes a decision, and how well it performs on our current data. Reported figures are from a synthetic corpus and live browser captures; real-world data collection is the stated next step.

1 The approach, in brief

The unit of analysis is the **update**, not the extension. Given a trusted baseline version (v1) and a candidate updated version (v2) of the same extension, the system runs each version inside a sealed, headless browser driven through an identical scripted user session against **byte-identical pages**. Because the input is held fixed, any difference in observed behaviour is attributable to the extension version itself rather than to the variability of the live web. Each run is reduced to a behavioural record across four channels — network requests, DOM access, storage/cookie access, and extension-API calls — and the two records are subtracted to form the **behavioural delta**, written as $\Delta = f(v2) - f(v1)$. The delta is then scored, and an explainable verdict is produced.

Scoring uses three complementary signals: a set of **transparent rules** derived from the security literature (the interpretable control), a **machine-learning model** that learns to weigh and combine the delta features, and an **anomaly detector** that flags updates whose behaviour is unusual relative to ordinary benign updates. Agreement between the signals is itself reported as a measure of confidence.

2 Figure 1 — The system approach (end-to-end pipeline)

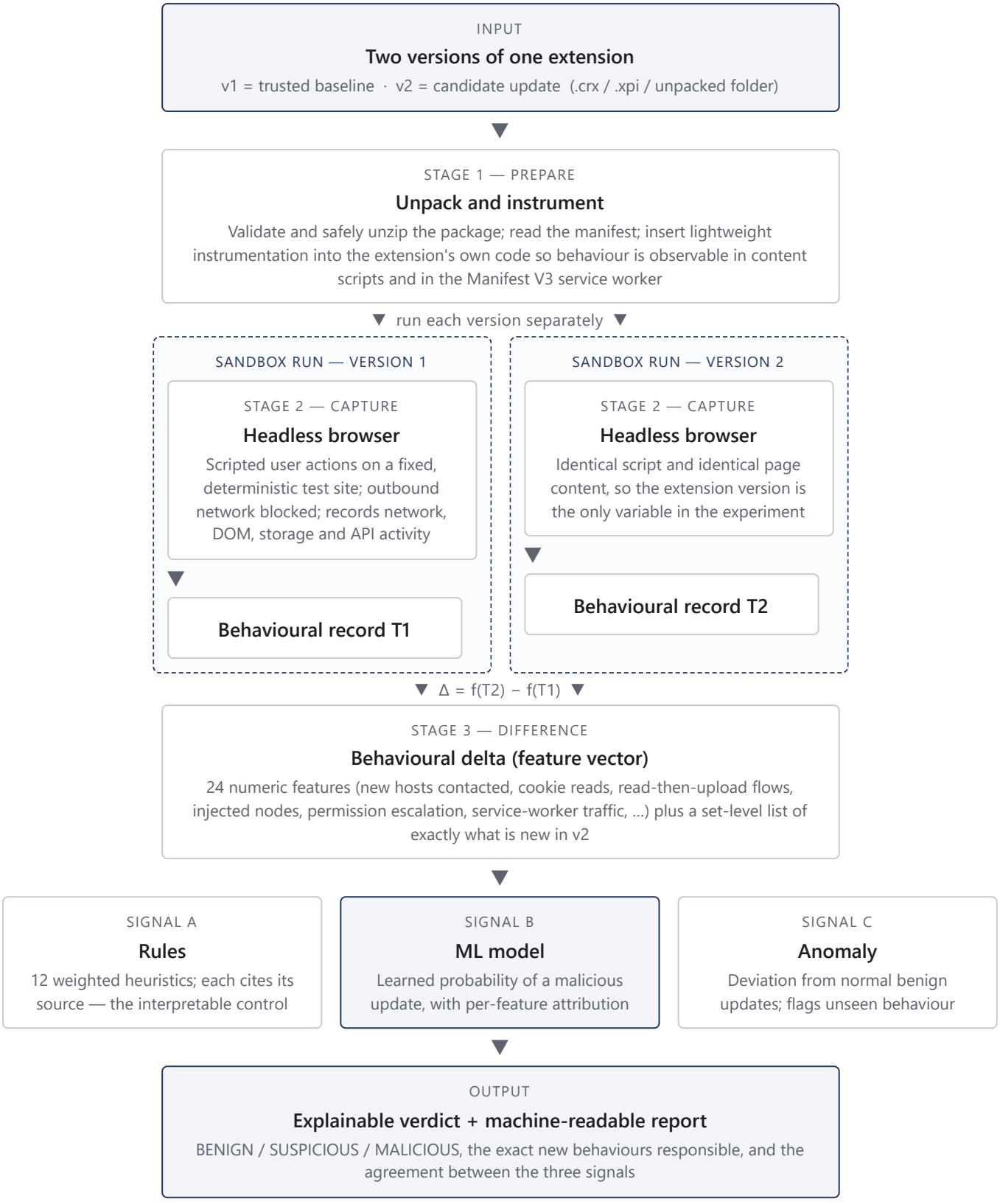


Figure 1. The extdrift pipeline. The extension version is the only difference between the two runs, which is what makes the behavioural delta meaningful.

3 Figure 2 — How the machine-learning model works

The model is developed in two phases: a one-time **training** phase that produces a fitted model from labelled examples, and an **inference** phase that applies the fitted model to a new update. A separate **anomaly** path handles behaviour the model has never been trained on.

3a. Training — producing the model

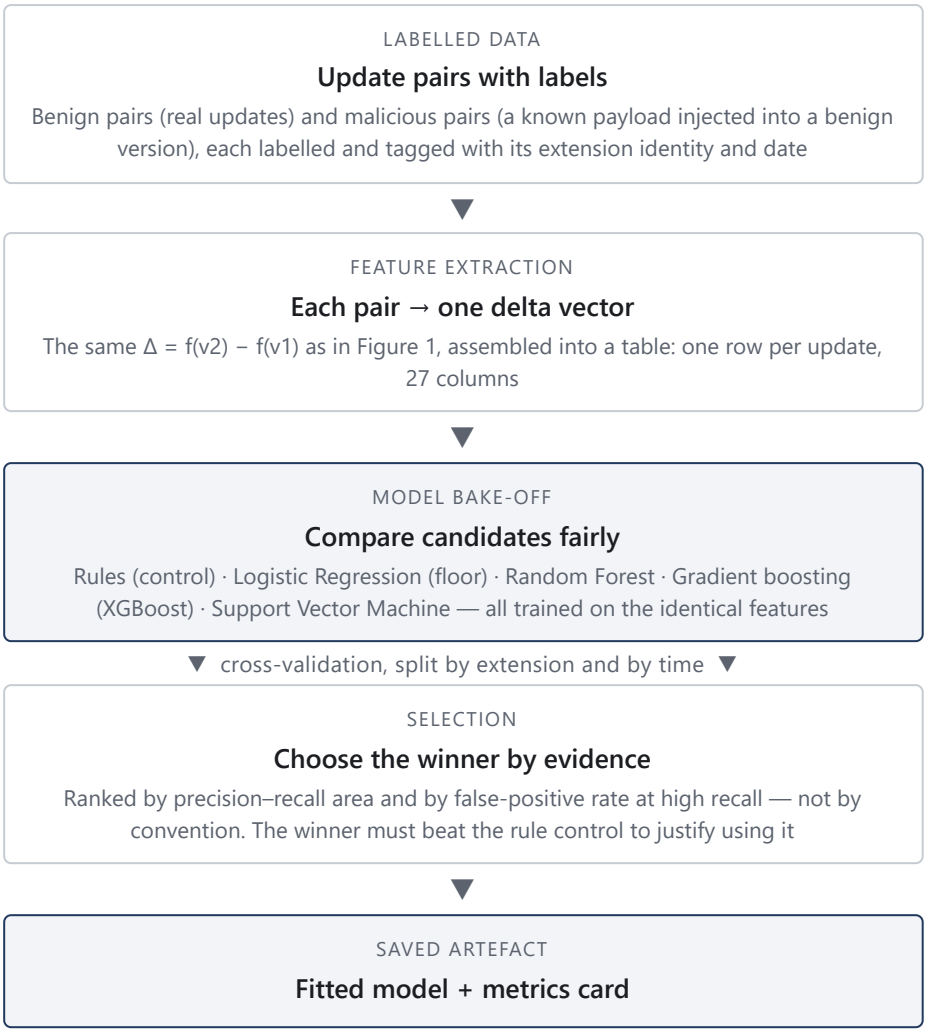


Figure 2a. Training. The model that is used is decided experimentally, not assumed in advance.

3b. Inference — deciding on a new update



Figure 2b. Inference. The output is decision support for a reviewer — a ranked, explained flag — not an automated takedown.

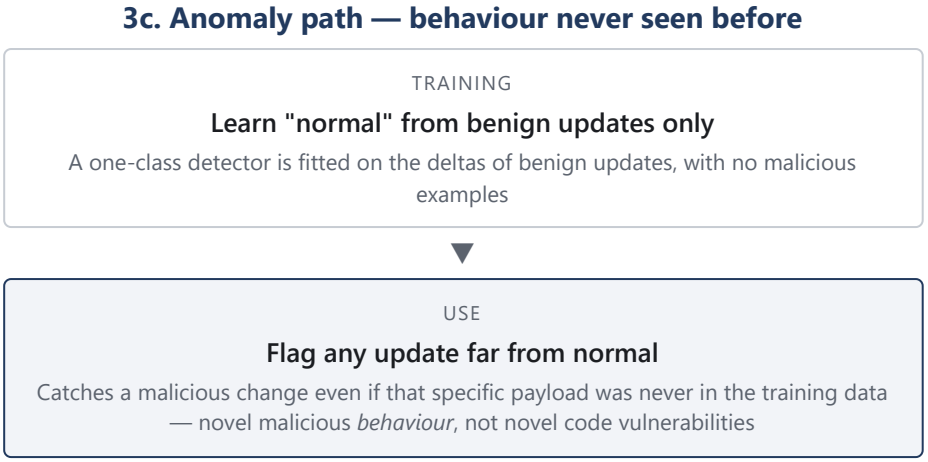


Figure 2c. The anomaly path is how the system reasons about genuinely unseen threats, without over-claiming.

> **What the model predicts, precisely.** Given the delta vector for an update, it outputs the probability that the update introduced malicious behavioural change. It is deliberately *not* a judgement of whether an extension is "good" or "bad" in general, and it does not attempt to discover unknown code vulnerabilities. This narrow, well-defined target is what makes the results meaningful and defensible.

4 How useful the model is — current evaluation

The following figures come from a synthetic corpus of **180 update pairs across 60 distinct extensions** (with six families of injected malicious behaviour), evaluated with cross-validation that keeps all versions of any one extension within a single fold — so the model cannot simply memorise an extension. The transparent rules are included as a baseline.

Model	PR-AUC	ROC-AUC	Precision	Recall	False-positive rate at 90% recall
Random Forest (selected)	0.999	0.999	1.000	0.948	0.00
Support Vector Machine	0.998	0.998	0.974	0.961	0.00
Logistic Regression	0.997	0.997	0.960	0.935	0.00
XGBoost (gradient boosting)	0.978	0.970	0.987	0.974	0.01
Rules (interpretable control)	0.868	0.884	0.632	0.870	0.66

The key comparison — why the learned model is worth having

The single most important metric for a detector of this kind is the false-positive rate: flagging harmless updates is what makes a system unusable in practice. To catch 90% of malicious updates, the transparent rules alone would also flag two out of every three *benign* updates. The learned model reaches the same detection level while flagging essentially none.



Unseen threats and real-pipeline confirmation

- **Unseen behaviour.** Trained only on benign updates and then tested against malicious families it had never seen, the anomaly detector flagged 100% of those held-out families while keeping the benign false-

positive rate near 6%. This demonstrates the system can react to novel malicious *behaviour*, which is the honest version of "detecting the unknown".

- **It works on the real pipeline.** A model trained only on synthetic data correctly judged our *live browser captures*: a genuine benign update scored below the decision threshold (BENIGN) and a weaponised update scored well above it (MALICIOUS), with the independent collector log confirming that data actually left the browser in the malicious case.

> **Honest reading of these numbers.** They are high because the current data is synthetic and therefore cleaner than the real world will be. The result that matters is not the absolute score but the *relationship*: the learned model reduces false positives dramatically relative to rules, and the anomaly layer generalises to unseen families. Confirming this on real-world update pairs is the next milestone.

5 Why this approach is different — and why it helps

Approach	Unit analysed	Resistant to obfuscation?	Detects a trusted extension turning malicious on update?
Store static review	One version	No — code can be hidden	No — the malicious update passes the same review
Static update-diff (prior work, CCS 2020)	Update, source code	No — authors note evasion	Partially, but blind to hidden code
Dynamic single-version (Hulk; ExtPrivA)	One version, behaviour	Yes	No — there is no version comparison
extdrift (this project)	Update, behaviour	Yes — we observe effects, not code	Yes — this is exactly what it scores

Obfuscation hides the *code*, but the *runtime effect* — for example reading a session cookie and then sending it to a host the previous version never contacted, from the background service worker — is still visible in the behavioural delta. Holding the page content fixed between the two runs removes the ordinary noise of the live web, which is what allows the false-positive rate to fall to a usable level.

6 Status and next steps

- **Implemented and running:** unpack, instrumentation, deterministic sandbox capture, the four-channel behavioural delta, the rule scorer, the synthetic-corpus generator, the dataset builder, the model bake-off, and the anomaly detector. The full test suite passes.
- **Next:** collect real benign update pairs (open-source release pairs, historical archives, and weekly self-snapshots) to validate the false-positive claim on real data; add a static code-diff baseline and run the obfuscation head-to-head that demonstrates the advantage over prior work; then a written evaluation.
- **Deliberately out of scope** (stated for honesty): consumer-side prevention before an update runs, discovery of unknown code vulnerabilities, and language-specific analysis — the evidence does not support claiming these.

---

extdrift · CSD493 Project-1 · Bind Pratap Singh & Suhani Deepak Agrawat · Advisor Dr. Sweta Mishra. Figures reflect a synthetic corpus and live captures; real-data evaluation is the next milestone. No real malware is executed outside the sealed, network-restricted sandbox.